

Diagrammi di Sequenza

Trentin Tommaso

November 26, 2025

Contents

1	Diagrammi UML Comportamentali	2
2	Diagrammi di Interazione	2
3	Diagrammi di Sequenza - Notazione	2
3.1	Diagramma di Sequenza	2
3.2	Attore	2
3.3	Partecipante / Istanza di classe	2
3.4	Lifeline	3
3.5	Messaggio	3
3.5.1	Scambio di Messaggi	3
3.5.2	Tipi	3
3.6	Activation Box	4
3.7	Messaggi di auto-delega / Activation box innestate	4
4	Dagli Scenari ai Diagrammi di sequenza al Codice OO	4
5	Paradigmi di Modellazione Centralizzati / Distribuiti	5
6	Creazione / Distruzione degli Oggetti	5
6.1	Creazione	5
6.2	Distruzione	5
7	Frame di interazione / Frammenti combinati	5
7.1	Limitazioni notazione base	5
7.2	Idea	6
7.3	Operatori frammento più usati	6
7.4	Pro e Contro	6
8	Quando utilizzare Sequence Diagrams	6
8.1	Analisi dei requisiti	6
8.2	Progettazione	6
8.3	Esempio	6

1 Diagrammi UML Comportamentali

Diagrammi che modellano comportamento dinamico degli elementi che compongono un sistema.

2 Diagrammi di Interazione

Diagrammi che modellano **collaborazione dinamica** tra oggetti che collettivamente implementano un determinato comportamento.

3 Diagrammi di Sequenza - Notazione

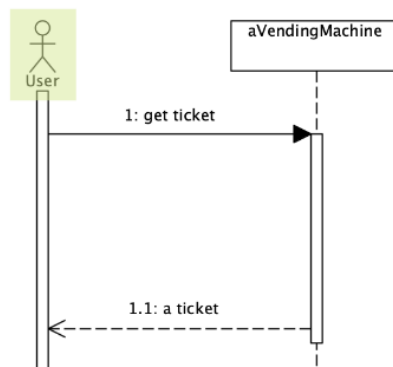
3.1 Diagramma di Sequenza

Modella interazioni tra 1+ attori e le parti del sistema SW, nell'ambito dell'esecuzione di uno o più scenari di un caso d'uso:

- Interazioni tra **attori** e **oggetti** del sistema sono modellate in forma di **messaggi** (OO paradigm);
- Scambio di messaggi viene letto dall'alto al basso;
- Chiariscono chiamate tra partecipanti e responsabilità di ciascuno di loro.

3.2 Attore

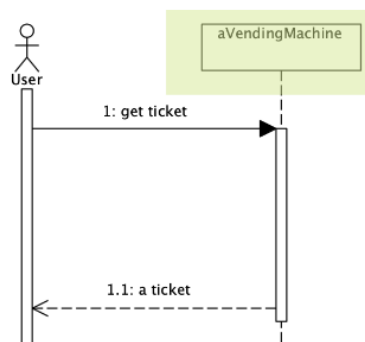
- Rappresentati come nel diagramma dei casi d'uso;
- Riportati a sinistra con frecce che modellano interazioni verso gli oggetti del sistema;
- Possono non essere presenti nel caso uno scenario venga avviato da un altro scenario o dal sistema stesso.



3.3 Partecipante / Istanza di classe

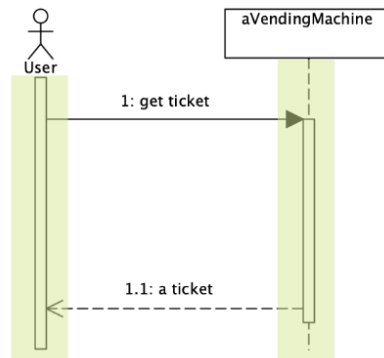
Istanze delle classi rappresentate da rettangoli etichettati in uno dei seguenti modi:

- con nome della classe e identificatore dell'oggetto nel formato **nomeOggetto:nomeClasse**;
- con nome che suggerisce che si considera un'istanza di una classe (anOrder, aProduct...).



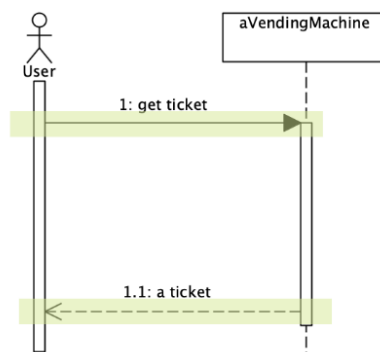
3.4 Lifeline

Mostrano i partecipanti mediante linea verticale tratteggiata dall'alto in basso che parte dai partecipanti, ne rappresentano il periodo temporale di vita (mostrano ordinamento messaggi, non durata temporale interazione!).



3.5 Messaggio

- Definisce particolare comunicazione tra le lifeline di un'interazione;
- Rappresentato mediante freccia tra lifeline (attore / oggetto o oggetto / oggetto);
- Ordine dei messaggi ricalca ordine sequenziale con il quale essi vengono scambiati.



3.5.1 Scambio di Messaggi

Un messaggio può:

- Invocare operazione (**chiamata**);
- Creare / distruggere istanza;
- Inviare segnale / dato.

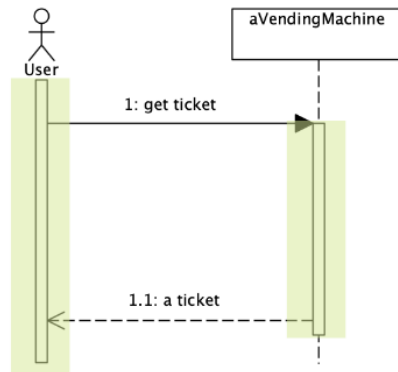
I messaggi possono invocare operazioni in modo sincrono / asincrono e, quando lifeline riceve messaggio di chiamata, deve esistere **corrispondente operazione nella classe della lifeline ricevente**.

3.5.2 Tipi

Rappresentazione Sender Receiver	Tipo	Semantica
	Messaggio Sincrono	Il chiamante attende che il ricevente completi
	Messaggio Asincrono	Il chiamante invia un segnale al ricevente e non attende il completamento
	Messaggio di ritorno	Il ricevente completa una chiamata sincrona e restituisce il controllo al chiamante
	Creazione	Il chiamante crea il target (invocando, ad esempio, un costruttore)
	Distruzione	Il chiamante distrugge il ricevente
	Messaggio trovato	Il messaggio è inviato esternamente all'ambito dell'interazione
	Messaggio perso	Il messaggio non raggiunge la destinazione (può modellare i casi di errore)

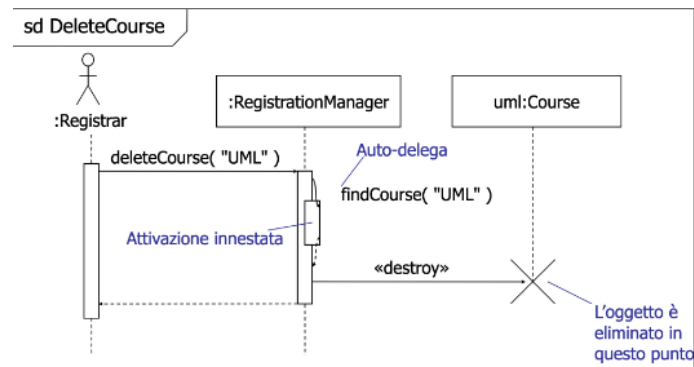
3.6 Activation Box

Rettangoli che coprono parte delle lifeline, rappresentano periodo in cui istanza è attiva nell'interazione. Lifeline di un partecipante si attiva quando esso elabora un messaggio: durante esecuzione l'attivazione si sposta tra le lifeline descrivendo flusso di controllo.



3.7 Messaggi di auto-delega / Activation box innestate

Auto-delega o chiamata interna quando un oggetto invia un messaggio a se stesso (chiamata a proprio metodo), ciò genera un'attivazione innestata.



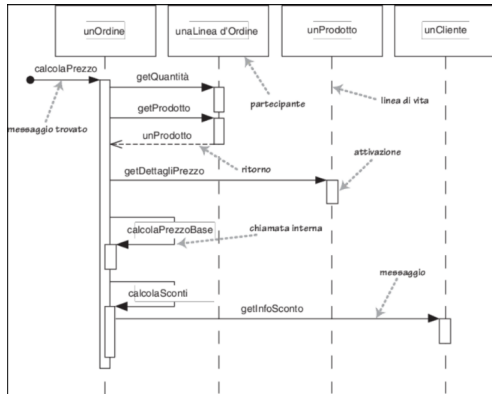
4 Dagli Scenari ai Diagrammi di sequenza al Codice OO

Vedi esempio da slide 16 su [10_sequence_diagrams.pdf](#)

5 Paradigmi di Modellazione Centralizzati / Distribuiti

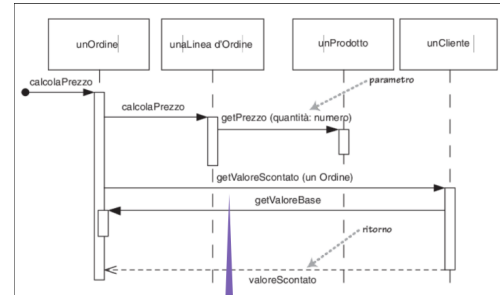
Centralizzato: un singolo oggetto ha controllo, gli altri forniscono servizi.

- **Pro:** semplice soluzione che tende a raccogliere in un solo punto tutta l'elaborazione.
- **Contro:** logica che gestisce attributi potrebbe trovarsi in classi diverse, rendendo manutenzione non intuitiva.



Distribuito: responsabilità distribuite tra gli oggetti partecipanti.

- **Pro:** conforme a paradigma OO, facilita manutenzione raccogliendo nello stesso oggetto dati / logica che potrebbero essere modificati assieme (permette utilizzo polimorfismo al posto di logica condizionale).
- **Contro:** può risultare dispersivo / difficile da leggere durante l'analisi.



IN QUESTO CASO ABBIAMO MODELLATO ESPLICITAMENTE ANCHE UN PARAMETRO

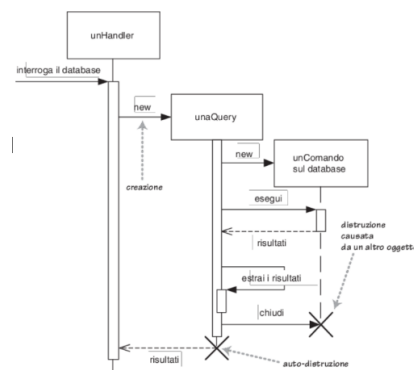
6 Creazione / Distruzione degli Oggetti

6.1 Creazione

Un messaggio al costruttore (etichettato come **new**) di un oggetto può creare un oggetto che viene disegnato al termine della freccia corrispondente al messaggio. Oggetti staticamente definiti partono dalla cima del diagramma e, in caso il partecipante faccia qualcosa appena creato, la activation box si disegna attaccata al rettangolo.

6.2 Distruzione

Se freccia del messaggio tra oggetti termina con una X allora oggetto che invia il messaggio sta distruggendo quello che lo riceve, una X alla fine della lifeline, senza messaggio, indica che l'oggetto sta distruggendo se stesso. Una volta distrutto un oggetto non è più disponibile per l'elaborazione.



7 Frame di interazione / Frammenti combinati

7.1 Limitazioni notazione base

- Diagrammi di sequenza sono più adatti a descrivere interazione tra oggetti che la logica di controllo;
- Notazione base non permette di rappresentare comportamenti ciclici / condizionali;
- UML2 implementa frame e frammenti con i loro operatori per ovviare a questi limiti.

7.2 Idea

Frame di interazione evidenziano una parte del diagramma e racchiudono sottosequenza di messaggi, possono essere composti da 1+ frammenti combinati. Ogni frame ha:

- **1 operatore** che determina come vengono eseguiti suoi operandi;
- **1+ frammenti**;
- **0+ condizioni di guardia** che stabiliscono se frammenti corrispondenti devono essere eseguiti

7.3 Operatori frammento più usati

Operatore	Nome completo	Semantica
opt	Optional	Il frammento corrispondente è eseguito solo se la sua condizione è vera. Costrutto if-then
alt	Alternatives	Molteplici frammenti in alternativa. Viene eseguito solo il frammento la cui condizione è vera. Ammette else come condizione di guardia. Costrutto if-then-else
loop	Loop	Ciclo. Il frammento è eseguito finché la condizione di guardia è verificata. La condizione è indicata all'inizio (while-do) o alla fine (do-while)
ref	Reference	Il frame rimanda ad un'interazione definita in un altro diagramma. Possono essere indicati dei parametri ed un tipo di ritorno. Nota: il frame deve racchiudere tutte le lifeline coinvolte nell'interazione

7.4 Pro e Contro

- **Pro:**
 - permettono di rappresentare costrutti condizionali / iterativi;
 - permettono di modellare sia scenario principale che quelli alternativi in un caso d'uso.
- **Contro:**
 - frame appesantiscono il diagramma;
 - poco adatti a rappresentare algoritmi complessi.

8 Quando utilizzare Sequence Diagrams

I sequence diagram sono utilizzati in diverse fasi del ciclo di vita di un SW.

8.1 Analisi dei requisiti

In fase di analisi, un sequence diagram può essere rappresentazione grafica di uno scenario di un caso d'uso. Il ruolo degli oggetti sarà ricoperto da un generico oggetto Sistema e potranno poi comparire altri oggetti se essi rappresentano attori o elementi vincolati nei requisiti.

8.2 Progettazione

In fase di progettazione i sequence diagram descrivono sempre scenari di casi d'uso. I partecipanti sono istanze delle classi del sistema mentre i messaggi sono responsabilità / operazioni stabilite dal diagramma delle classi.